

How to Use This Preparation Sheet

This preparation sheet replaces the usual lecture video. Work through it at home; we will discuss it together at the beginning of the next session before starting Worksheet 12.

TL;DR: Learn the fundamentals of null-terminated strings so that you understand the problem we will solve in Worksheet 12.

Learning Objectives

- Understand how null-terminated strings are represented and processed.
- Revisit the relationship between arrays and pointers.
- Understand how `strlen`, `strcpy`, and `strcmp` operate and what their costs are.
- Recognize the limitations of fixed-size arrays and character-by-character string operations.

Motivation

In Worksheet 9, we implemented a symbol table for our compiler. For simplicity, we represented identifiers using

```
type String: array[42] of char;
```

This worked, but it was deliberately only a temporary solution.

Before we can choose a better representation, we need to understand more precisely how strings are commonly represented and processed in ABC (and C). In particular, we will look at null-terminated strings and operations such as determining their length, copying them, and comparing them.

This will reveal two issues that matter for our symbol table. First, fixed-size arrays are a poor fit for strings of different lengths: choosing a large array wastes memory, while choosing a small one limits the strings we can store. Second, operations such as comparing two strings may require inspecting their characters one by one.

This preparation sheet therefore focuses on the fundamentals of null-terminated strings. In Worksheet 12, we will use what we learn here to develop a string representation that is better suited to the particular requirements of our compiler.

Remark: There are other important use cases with very different requirements. For example, a lexer has to construct strings incrementally while reading characters. A dynamically growing string based on `realloc` would be useful for that purpose. We will need something like this later, but it is deliberately not the data structure we develop in this worksheet.

Step 1 — Strings and Arrays (Review)

[Prepare as homework]

In Worksheet 9, we used the following type to represent strings:

```
type String: array[42] of char;
```

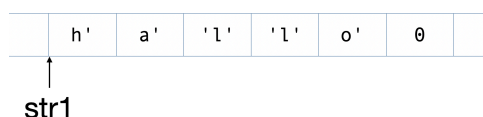
This was convenient because every variable of type `String` had the same type and enough space for many of the identifiers we wanted to use. But the choice of `42` was arbitrary. Before we look for a better representation, let us first review what a string actually is in ABC (and C).

A string is a sequence of characters terminated by a null character. To store such a string in an array, the array must therefore have enough space for all characters and the terminating null character.

For example, the string `"hallo"` can be stored in the array `str1` as follows:

```
global str1: array[6] of char = {'h', 'a', 'l', 'l', 'o', 0};
```

The string consists of five characters plus the terminating null character. We can describe the corresponding memory as follows:



In the initializer list above, all six array elements were initialized explicitly. Recall that if fewer initializers than array elements are given, the remaining elements are initialized with zero. We could therefore equivalently write

```
global str1: array[6] of char = {'h', 'a', 'l', 'l', 'o'};
```

If an array is initialized as part of its declaration, its dimension may also be omitted. The compiler then determines the dimension from the initializer. For example,

```
global str1: array[] of char = {'h', 'a', 'l', 'l', 'o', 0};
```

creates an array with dimension 6.

Whereas

```
lobal str1: array[] of char = {'h', 'a', 'l', 'l', 'o'};
```

creates an array with dimension 5. In the second case, no null character is stored in the array, and the array therefore does not contain a valid null-terminated string.

A string literal can also be used as an initializer:

```
global str2: array[] of char = "hello";
```

For an array initializer, the compiler treats the string literal as if all characters including the terminating null character had been listed explicitly:

```
global str2: array[] of char = {'h', 'e', 'l', 'l', 'o', 0};
```

The dimension of `str2` is therefore 6. Consequently,

```
global str2: array[5] of char = "hello";
```

cannot work: the array is too small to hold the complete string including its terminating null character.

Tasks

Create the following program `ex1.abc`:

```
@ <stdio.h>

global str1: array[] of char = {'h', 'a', 'l', 'l', 'o'};
global str2: array[5] of char = "hello";

fn main()
{
    printf("str1 = %s\n", str1);
    printf("str2 = %s\n", str2);
}
```

1. The compiler should complain that the dimension of `str2` is too small. Confirm this first. Then change its dimension to 6.
2. **Undefined behavior:** On my machine, the output looks like this:

```
MCL:s22 lehn$ ./a.out
str1 = hallohello
str2 = hello
```

Why did the first call to `printf` print both character sequences on my machine? Draw a sketch of the relevant memory contents and use it to explain the observed output.

Important: The particular output above is not guaranteed — the program has undefined behavior.

3. Add

```
printf("sizeof(str1) = %zu\n", sizeof(str1));
printf("sizeof(str2) = %zu\n", sizeof(str2));
```

to `main`. Now change the dimension of `str1` to 42 and omit the dimension of `str2`. Compare the results.

What does `sizeof` tell us here: the size of the string, or the size of the array used to store it?

4. The standard-library header `<string.h>` declares the function `strlen`, which determines the length of a null-terminated string. Include the header with

```
@ <string.h>
```

and add

```
printf("strlen(str1) = %zu\n", strlen(str1));
printf("strlen(str2) = %zu\n", strlen(str2));
```

to `main`.

Compare the values obtained with `sizeof` and `strlen`. Notice that the terminating null character is not counted as part of the string length.

Finally, determine what happens when the dimension of `str1` is changed to 5 or omitted completely. Explain why calling `strlen(str1)` is problematic in this case.

Step 2 — Strings and Pointers (Review)**[Prepare as homework]**

Arrays and pointers can often be used in similar ways, although they are **different data types**. Consider the following declarations:

```
global str1: array[] of char = "hallo";
global str2: -> char = str1;
```

Here, `str1` is an array containing the characters of the string, whereas `str2` is a pointer. The initialization of `str2` is equivalent to

```
global str2: -> char = &str1[0];
```

because an array can be implicitly converted to a pointer to its first element.

We can therefore use both `str1` and `str2` when a pointer to the first character of the string is required. However, this does not mean that `str1` and `str2` are the same thing:

- `str1` contains the characters.
- `str2` only contains the address at which these characters can be found.

Tasks

1. Consider the following program:

```
@ <string.h>
@ <stdio.h>

global str1: array[] of char = "hallo";
global str2: -> char = &str1[0];

fn main()
{
    printf("sizeof(str1) = %zu\n", sizeof(str1));
    printf("sizeof(str2) = %zu\n", sizeof(str2));

    printf("strlen(str1) = %zu\n", strlen(str1));
    printf("strlen(str2) = %zu\n", strlen(str2));

    printf("str1 = %s\n", str1);
    printf("str2 = %s\n", str2);
}
```

Before compiling the program, draw a sketch of the memory occupied by `str1` and `str2`. In particular, indicate what is stored in `str1` and what is stored in `str2`.

2. Compile and run the program. Compare the results of `sizeof` and `strlen`. Explain why

`strlen(str1) == strlen(str2)`

but

`sizeof(str1) != sizeof(str2)`

on our system. What information about the string is available from `str2` itself?

Takeaway: A pointer to a string tells us where the characters begin, but not how many characters are stored there or how large the underlying memory region is.

Step 3 — Call by Value**[Prepare as homework]**

In ABC, function arguments are passed by value: the function receives a copy of the argument. This also applies to arrays. In C, arrays cannot be passed as function arguments by value for historical reasons. When an array is used as an argument, a pointer to its first element is passed instead. The following example illustrates the important difference between passing an array and passing a pointer in ABC:

```
@ <stdio.h>

global str: array[42] of char = "hallo";

fn foo(str: array[42] of char)
{
    str[0] = 'X';
    printf("in foo: str = %s\n", str);
}

fn bar(str: -> char)
{
    str[0] = 'X';
    printf("in bar: str = %s\n", str);
}

fn main()
{
    printf("in main: str = %s\n", str);
    foo(str);
    printf("in main: str = %s\n", str);
    bar(str);
    printf("in main: str = %s\n", str);
}
```

Tasks

1. Before running the program, predict its output. Explain to your neighbor why the global variable `str` is unchanged after the call to `foo`, but is modified after the call to `bar`. In particular, think about what is copied when each function is called.
2. Explain to your neighbor why calling `foo` is more expensive than calling `bar`. How many bytes have to be copied for the argument of `foo`? How many bytes have to be copied for the argument of `bar` on our 64-bit system?
3. Now imagine that instead of `array[42] of char` we had chosen `array[1024] of char` as our general representation of strings. What would happen to the cost of calling `foo`? What would happen to the cost of calling `bar`?

Step 4 – String Functions**[Prepare as homework]**

In `my_string.abc` we deliberately start with the fixed-size representation from Worksheet 9:

```
@ <stdio.h>

type String: array[42] of char;

fn my_strlen(str: String): size_t
{
    local len: size_t = 0;
    while (str[len]) {
        ++len;
    }
    return len;
}

fn main()
{
    printf("%zu\n", my_strlen("hallo"));
    printf("%zu\n", my_strlen("hi"));
    printf("%zu\n", my_strlen(""));
}
```

However, when passing a string to a function, we usually do not pass the entire array containing the string. Instead, we pass a pointer to its first character. This has several advantages:

1. **Passing the argument is cheap.** Only a pointer — 8 bytes on our system — has to be copied, independent of the length of the string or the size of the array containing it.
2. **Strings of different lengths can be handled by the same function.** With an array parameter, the array dimension would be part of the parameter type. We would therefore have to choose some fixed maximum size such as 42.
3. **The function can access the original characters.** If it is supposed to modify them, we pass a pointer to `char`. If it must not modify them, we pass a pointer to `readonly char`.

Tasks

1. Try to compile the program in its current form. You should get an error similar to

```
ex4.abc:14.31-14.38: error: can not convert an expression of type 'array [6] of u8' to
type 'array [42] of char'
```

Why does the call `my_strlen("hallo")` not match the parameter type `String`?

We can force it to match using an explicit cast:

```
printf("%zu\n", my_strlen((String)"hallo"));
```

Change all three calls accordingly and compile the program again.

This works, but think about what has to happen: a temporary array of 42 characters is created from the string literal, and this array is then passed by value to `my_strlen`. We are doing quite a lot of copying just to determine the length of a string.

2. Explain to your neighbor how `my_strlen` determines the length of the string. How many characters have to be inspected for a string of length `n`?

Observation: The length is not stored anywhere. `my_strlen` has to search for the terminating null character. Its running time therefore grows linearly with the length of the string. This is essentially the same principle used by the standard-library function `strlen`.

3. We do not actually need a copy of the array to determine the string length. We only need to know where its first character is stored. Remove the definition of `String` and change the function declaration to

```
fn my_strlen(str: -> readonly char): size_t
```

Also remove the explicit casts from the calls. **Compile and run the program again. Notice that the implementation of `my_strlen` itself does not have to change at all.** Why can the same function now be called with strings of different lengths?

4. Suppose we call `my_strlen(str)` several times for the same long string. Does the second call know the result computed by the first call? How could a different string representation avoid searching for the null character every time?

Step 5 – Copying Strings**[Prepare as homework]**

The standard-library header `<string.h>` declares `strcpy` as follows:

```
extern fn strcpy(dst: -> char, src: -> readonly char): -> char;
```

The function copies the string beginning at `src` to the memory beginning at `dst`. The terminating null character is copied as well. Notice the parameter types:

- `src` is a pointer to `readonly char` because `strcpy` only reads the source string.
- `dst` is a pointer to `char` because `strcpy` modifies the memory it points to.
- The function returns `dst`, i.e. a pointer to the beginning of the copied string.

There is an important problem, however. As we saw in the previous step, a pointer tells us where some memory begins, but it does not tell us how large that memory region is.

Consequently, `strcpy` cannot determine whether there is enough space at `dst` to store the complete string. It simply copies one character after another, including the terminating null character. Making sure that enough memory is available is the responsibility of the programmer.

We will now implement in `mystrcpy.abc` our own version of `strcpy` — not because we can do it better, but because implementing it makes very clear what the function actually does.

```
@ <string.h>
@ <stdio.h>

/*
// TODO
fn my_strcpy(dest: -> char, src: -> readonly char): -> char
{
}
*/

fn main()
{
  local str: array[42] of char = "1234";

  strcpy(str, "hallo welt!");
  printf("%s\n", str);
}
```

Task

1. Compile and run the program. Verify that `"hallo welt!"` is copied into `str`.
2. Change the dimension of `str` from 42 to 5 and run the program again. The program now has undefined behavior. On my machine, I got a segmentation fault when compiling without optimizations, while with `-O3` the program appeared to work.
Explain why the behavior is undefined. In particular, determine how many bytes are available at `dst` and how many bytes `strcpy` attempts to write.
Why does the fact that the program appears to work with `-O3` not make the program correct?
3. Implement your own function `my_strcpy` and replace the call to `strcpy` in `main` by a call to `my_strcpy`.
Your function must copy all characters including the terminating null character and return `dst`. To make it easier to detect whether the terminating null character is copied correctly, change the declaration of `str` to

```
local str: array[] of char = "123456789";
```
4. Deliberately introduce a bug: modify `my_strcpy` so that it copies the characters but not the terminating null character.

Step 6 – Comparing Strings**[Prepare as homework]**

The standard-library header `<string.h>` declares `strcmp` as follows:

```
extern fn strcmp(s1: -> readonly char, s2: -> readonly char): int;
```

The function compares the two strings `s1` and `s2` lexicographically:

- If both strings are equal, `strcmp` returns 0.
- If `s1` comes before `s2`, it returns a negative value.
- If `s1` comes after `s2`, it returns a positive value.

Two strings are equal if they have the same length and all their characters are equal.

The following program demonstrates the use of `strcmp` and provides our own implementation called `my_strcmp`:

```
@ <string.h>
@ <stdio.h>

fn my_strcmp(s1: -> readonly char, s2: -> readonly char): int
{
  for (; *s1 && *s1 == *s2; ++s1, ++s2) {}
  return (int)*s1 - (int)*s2;
}

fn main()
{
  local str1: array[] of char = "abcX";
  local str2: array[] of char = "abcx";

  printf("strcmp(str1, str2) = %d\n", strcmp(str1, str2));
  printf("my_strcmp(str1, str2) = %d\n", my_strcmp(str1, str2));

  if (!strcmp(str1, str2)) {
    printf("strings are equal\n");
  } else {
    printf("strings are different\n");
  }
}
```

Task

Test the program with different strings. In particular, try strings that

- are equal,
- differ in their first character,
- differ only near the end,
- have different lengths.

Then explain to your neighbor (or to us):

1. How does `my_strcmp` work? In particular, under which conditions does the loop terminate?
2. Why are the casts in

```
return (int)*s1 - (int)*s2;
```

necessary?

Experiment: Remove the casts, observe what the function returns for different strings, and use your observations to figure out what must be happening.

3. How much work does `my_strcmp` have to do? Consider these two comparisons:

```
my_strcmp("Xbcdefghijklmnopqrstuvwxyz", "Ybcdefghijklmnopqrstuvwxyz")
and
```

```
my_strcmp("abcdefghijklmnopqrstuvwxyzX", "abcdefghijklmnopqrstuvwxyzY")
```

In which case can `my_strcmp` determine the result earlier? Why?

4. Recall our symbol table from Worksheet 9. To find an identifier, we traversed a linked list and compared the identifier stored in each node with the identifier we were looking for:

```
strcmp(s1, s2) == 0
```

Suppose our symbol table contains many identifiers. Why might string comparison become an operation worth optimizing?